

AtliQ Commerce

End-to-End Batch Data Engineering Project

Detailed Implementation, Architecture, Issues & Learning Report

Item	Details
Project type	End-to-end batch data engineering capstone
Cloud stack	Azure + Azure Data Factory + Azure Databricks + Microsoft Fabric
Transformation	PySpark / Databricks + dbt Core
BI	Microsoft Fabric / Power BI
Author	Jaideep Gupta
Report status	Reconciled against final README — reflects actual, verified state

Table of Contents

1. Executive Summary
2. Project Milestones
3. End-to-End Architecture
4. Source Data and Integration
5. M1 — OLTP Layer
6. M2 — Azure Data Factory
7. M3 — Databricks Bronze and Silver
8. M4 — dbt Gold
9. M5 — Nightly Automation
10. M6 — Microsoft Fabric
11. M7 — Git, CI/CD and Reliability
12. Issues Faced and Resolutions
13. Key Technical Learnings
14. Repository Evidence Structure
15. Known Gaps and Remaining Work
16. Interview / Client Explanation
17. Final Note

1. Executive Summary

This project implements a complete batch data engineering solution for AtliQ Commerce. Operational commerce data is sourced from Azure SQL, ingested through Azure Data Factory into ADLS Gen2, processed through Bronze and Silver in Azure Databricks, transformed into business-ready Gold models using dbt, automated as a nightly flow, and exposed to Microsoft Fabric for analytics.

The architecture intentionally separates OLTP from analytical processing. Azure SQL remains the operational source of truth, while the downstream lakehouse/analytical layers provide durable, transformed and reporting-oriented data.

All core engineering milestones (M1–M7) have been implemented and validated, including a verified two-run idempotency proof. Two items were deliberately scoped out rather than left as unverified claims: a “New vs. Returning Customers” dashboard view, and automated failure alerting on the nightly job. Both are documented as explicit, intentional gaps rather than presented as complete — see Section 15.

2. Project Milestones

Milestone	Work Completed	Status
M1 — OLTP	Azure SQL schema, ETL control/watermark table, daily transaction simulator	Done
M2 — Ingestion	Metadata-driven ADF SQL/CSV ingestion and orchestration via control table	Done
M3 — Silver	Databricks Bronze/Silver processing and incremental facts	Done
M4 — Gold/dbt	dbt staging/intermediate/mart models and tests (13/13 models, 18/18 tests)	Done
M5 — Nightly sync	Master orchestration, 2 AM trigger, retry/idempotency design	Done — two-run proof captured
M6 — Fabric	Gold availability and executive dashboard	Done, with one visual deferred (see §15)
M7 — CI/CD	GitHub repository, GitHub Actions, dbt CI, pipeline audit logging	Done — alerting deferred (see §15)

3. End-to-End Architecture

Azure SQL OLTP + CSV files → Azure Data Factory → ADLS Gen2 / Bronze landing → Azure Databricks (Bronze / Silver) → dbt (Gold analytical models) → Microsoft Fabric / Power BI

ADF orchestration uses `pl_master_batch`, with `pl_sql_to_raw` → `pl_sql_to_adls` and the downstream `Databricks_Bronze_to_Gold` job.

Why the layers exist

- OLTP: optimized for operational transactions and source-system integrity.
- Bronze: durable landing boundary that preserves source-oriented data before transformation.
- Silver: cleansing, typing, standardization, joins and enrichment.
- Gold: business-ready dimensions, facts and marts.
- Fabric/Power BI: consumes curated analytical data rather than repeatedly querying the operational source.

4. Source Data and Integration

4.1 Azure SQL

- customers
- products
- orders
- order_items
- payments

4.2 CSV Business Datasets

- `marketing_spend.csv`: `spend_date`, `channel`, `campaign`, `spend_amount`, `clicks`.
- `supplier_price_list.csv`: `product_id`, `product_name`, `supplier_name`, `supplier_cost`, `effective_date`.

The SQL data and the two business datasets were incorporated into the downstream analytical solution. Supplier pricing enriches product/sales information through `product_id`. Marketing spend is retained as a separate fact/mart because there is no sale-to-campaign attribution key; a direct date-only sales/marketing join could multiply rows and inflate metrics.

5. M1 — OLTP Layer

The operational database is Azure SQL database `atliq_commerce` on Azure SQL Server `atliq-sql-jd`.

- Created customers, products, orders, `order_items` and payments.
- Created an ETL control/watermark mechanism for incremental extraction.
- Implemented a daily transaction simulator to create changing source data for pipeline testing.

This establishes a realistic operational source and allows the downstream pipeline to demonstrate incremental processing rather than only a one-time historical load.

Open item: Explicit seed row-count verification (expected counts across customers/products/orders/order_items/payments) and a captured run of the daily simulator are tracked as an open evidence item — see Section 15.

6. M2 — Azure Data Factory

ADF is used for ingestion and orchestration.

Pipelines

- pl_sql_to_raw — SQL extraction/landing.
- pl_sql_to_adls — lands/moves data into ADLS-oriented Bronze storage.
- pl_master_batch — master orchestration; also records pipeline execution status through an Azure SQL audit procedure.

The ingestion is metadata-driven rather than hard-coded as one pipeline per source table: an ETL control table defines the source table, load type and watermark information, and the master flow uses that metadata to process each configured table.

- Read source data using the ingestion/control logic.
- Land the data into ADLS Gen2.
- Trigger downstream Databricks processing.
- Monitor pipeline runs for success/failure and duration.

ADF pipeline JSON exports and screenshots are stored as M2 repository evidence (evidence/M2_ADF/).

7. M3 — Databricks Bronze and Silver

Azure Databricks provides distributed transformation. Unity Catalog organizes the project assets under catalog atliq, including bronze, silver, gold, ci and fabric_gold schemas.

- Dimensions are rebuilt/overwritten where appropriate for a clean current state.
- Incremental Silver logic uses business-key MERGE: orders and payments deduplicate using updated_at DESC, while order_items is insert-only and uses created_at because line items do not change after creation.
- run_date supports batch traceability.
- Product and supplier information is incorporated during enrichment.

Fact Grain

fact_sales is one row per order item. Key fields include order_item_id, order_id, customer_id, product_id, order_date, status, quantity, item_price, gross_revenue, product/category information and supplier cost.

8. M4 — dbt Gold

dbt Core manages SQL transformations, model dependencies, testing and documentation while Databricks provides the analytical execution/storage environment.

Model Structure

- Staging: stg_customers, stg_marketing_spend, stg_order_items, stg_orders, stg_payments, stg_products, stg_supplier_price_list.
- Intermediate: int_sales_enriched.
- Mart: dim_customer, dim_date, dim_product, fact_sales, fact_marketing_spend.

The current physical implementation materializes staging and intermediate models as views in atliq.gold and mart models as tables. The file dim_products.sql produces the dim_product model.

Validation Results

- dbt ls: 13 models, 18 data tests, 7 sources.
- dbt run: successful; 13/13 models passed.
- dbt test: 18/18 tests passed; 0 warnings, 0 errors, 0 skips.
- Tests include not-null, uniqueness and relationship checks.

Validation	Result
Customer duplicate check	0 duplicates
Product duplicate check	0 duplicates
Fact order-item uniqueness	0 duplicates
Date referential integrity	0 orphan rows
Supplier cost completeness	798 / 798
Gross revenue	2,126,260.00
Supplier cost (quantity × supplier_cost)	1,363,271.87
Gross profit (revenue – supplier cost)	762,988.13
Profit reconciliation difference	0.00

Correction: This table is the single source of truth for these figures. An earlier draft of this report showed a different supplier-cost total (656,785.28); that value was a reconciliation error and has been superseded by the verified figure above, which also matches the idempotency proof in Section 9.

9. M5 — Nightly Automation

The nightly process ingests changed data, processes Silver incrementally, builds Gold and completes automatically.

- Master pipeline: pl_master_batch.
- Databricks job: Databricks_Bronze_to_Gold.

- Nightly trigger is configured on pl_sql_to_raw at 2:00 AM.
- The end-to-end master execution was successfully verified, including SQL-to-raw, SQL-to-ADLS and Databricks transformation.

Idempotency Proof

The complete pl_master_batch pipeline was executed twice against the same source state. Results were captured from atliq.gold.fact_sales after each full execution:

Metric	Run 1	Run 2	Result
fact_sales row count	798	798	Match
Total gross_revenue	2,126,260.00	2,126,260.00	Match
Total supplier cost of sold units	1,363,271.87	1,363,271.87	Match

Rerunning the same batch does not create duplicate sales records or change the calculated financial totals, providing evidence of idempotent/retry-safe processing. Supporting screenshots are stored in evidence/M5_NIGHTLY_SYNC/.

10. M6 — Microsoft Fabric

Curated Gold data is made available to Fabric through the project's Fabric/OneLake integration path; the Fabric semantic model is built over Gold rather than querying Azure SQL directly. The executive dashboard includes:

- Revenue trend by month.
- Top products by revenue.
- Top cities by revenue.
- Revenue by category.

Revenue definition: gross revenue excludes Cancelled orders. Returned activity is retained separately through order status so it is not silently netted out of, or inflating, the sales measure.

Known gap: A “New vs. Returning Customers” view (signup cohort vs. first/repeat purchase month) was planned but is not yet built. It is tracked as a follow-up item in Section 15 rather than presented as delivered.

11. M7 — Git, CI/CD and Reliability

The project is version-controlled in GitHub. GitHub Actions validates dbt changes.

- Repository contains the implementation and dbt project.
- GitHub Actions runs dbt CI for pull-request validation.
- Databricks host, HTTP path and token are supplied through GitHub Secrets.
- The latest CI run succeeded after the Databricks token secret was updated.

Pipeline Audit Logging

The ADF master pipeline includes Audit Start and Audit End stored-procedure activities that write execution metadata (pipeline name, run ID, start/end timestamps, status) to the Azure SQL audit table via [etl].[usp_log_pipeline_audit].

Scope decision: Automated failure alerting (email/webhook notification on pipeline failure) was evaluated and intentionally not implemented in this version. Run status is available in the audit table for manual review; active alerting is listed as a future enhancement in Section 15.

At the time evidence packaging began, git status showed a clean working tree because the newly created evidence directories were empty; Git does not track empty directories.

12. Issues Faced and Resolutions

Issue	What Happened	Resolution / Learning
PowerShell execution policy	Script execution was blocked during local dbt setup.	Used process-scoped Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned.
dbt-Databricks authentication	Initial dbt connection did not work.	Configured environment variables in profiles.yml; corrected the host to omit https:// and replaced the token.
Credential exposure risk	A Databricks token was accidentally pasted during troubleshooting.	Treated it as compromised and replaced it. Current design uses environment variables/GitHub Secrets.
GitHub Actions failure	CI initially failed because the Databricks authentication secret was stale/invalid.	Updated DATABRICKS_TOKEN in GitHub Secrets and reran CI successfully.
Catalog naming	Correct catalog name is atliq; a spelling mismatch can cause object-not-found errors.	Standardized the project on atliq.
Sales + marketing integration	A date-only join could multiply sales rows because campaigns/channels are not uniquely attributable to orders.	Kept marketing spend as a separate fact/mart instead of creating misleading attribution.
Evidence folders	Git reported clean status while evidence folders were empty.	Screenshots/JSON were collected before git add/commit.
Supplier cost reconciliation	An earlier draft reported a supplier-cost figure (656,785.28) that did not match the value computed from fact_sales.	Recomputed quantity × supplier_cost directly from the Gold fact table; corrected figure (1,363,271.87) now

Issue	What Happened	Resolution / Learning
		matches across all report sections and the idempotency proof.

13. Key Technical Learnings

- ADF handles ingestion/orchestration; Databricks handles distributed transformation.
- ADLS Gen2 provides durable cloud storage independent of compute availability.
- Unity Catalog organizes and governs data assets.
- Bronze is source-oriented, Silver is standardized/enriched, and Gold is business-facing.
- dbt manages SQL transformation dependencies, tests and documentation; it does not replace the Databricks storage/compute platform.
- Fact grain must be explicit: fact_sales is one row per order item.
- MERGE is important for incremental processing and retry safety; orders/payments use updated_at for latest-record handling, while order_items is append-only and uses created_at.
- Business metric definitions must be explicit; Cancelled orders are excluded from revenue in this project.
- Marketing attribution requires a valid attribution key.
- Secrets should never be hard-coded or committed.
- Financial totals reported in more than one place must be reconciled against the same query — a mismatched figure is a correctness bug, not a documentation nit.
- Disclosing a deliberately unbuilt feature is preferable to silently omitting it or overstating it as done.

14. Repository Evidence Structure

```
. ├── .github/ | └─ workflows/ | └─ ci.yml | sql/ | └─ 01_schema_ddl.sql |
└─ 07_etl_control_table.sql | └─ 08_pipeline_audit.sql | dbt_project/ | └─ models/ |
└─ tests/ | └─ macros/ | └─ analyses/ | └─ dbt_project.yml | └─ profiles.yml |
evidence/ | └─ M1_OLTP/ | └─ M2_ADF/ | └─ M3_SILVER/ | └─ M4_GOLD_DBT/ | └─
M5_NIGHTLY_SYNC/ | └─ M6_FABRIC/ | └─ M7_CICD/ | atliq_commerce_architecture.svg |
Cdebasics_AtliQ..._Report.pdf / .docx | .gitignore | README.md
```

Alignment note: This structure now matches the repository's actual layout and the project README exactly (previous drafts of this report referenced a Soln/ and docs/ layout that no longer applies).

15. Known Gaps and Remaining Work

The following are the only open items against the project guide. Everything else described in this report has been implemented and evidenced.

- ☐ Capture explicit OLTP seed row-count verification and a recorded run of the daily transaction simulator (M1).

- ☐ Build the “New vs. Returning Customers” dashboard view (customer cohort by signup month vs. first/repeat purchase month) (M6).
- ☐ Decide finally on automated failure alerting for the nightly job — currently intentionally deferred; audit-table logging is in place as the interim control (M7).
- ☐ Review all evidence files for secrets before committing.
- ☐ Run git status, inspect the exact files, then git add/commit/push.
- ☐ Perform a final submission audit against the project guide, cross-checking this report against README.md for consistency.

16. Interview / Client Explanation

I built an end-to-end nightly batch data engineering pipeline for an e-commerce workload. Azure SQL acts as the OLTP source. Azure Data Factory performs metadata-driven ingestion and orchestration off a control table, landing data into ADLS Gen2. Azure Databricks processes the data through Bronze and Silver, using incremental MERGE logic for the sales fact. dbt Core builds and tests the Gold analytical models in Databricks. The curated data is exposed to Microsoft Fabric for BI. The solution is version-controlled in Git, with GitHub Actions validating dbt changes and an audit table logging every pipeline run.

I also validated data quality, financial reconciliation and retry/idempotency behavior with an explicit two-run proof, so the pipeline is repeatable and production-oriented. Two enhancements — a customer-cohort dashboard view and automated failure alerting — were scoped out for this iteration; I can speak to how I'd design and prioritize them next.

17. Final Note

This report is a learning and implementation record based on the supplied project guide and the work completed in the project. It has been reconciled against the project README so that both documents present the same figures and the same completion status for every milestone. Where a guide requirement is genuinely not yet built, it is disclosed in Section 15 rather than presented as proven.